講義 4：Internal Table（內表：程式執行時放在記憶體裡的表格）基礎

> 對應練習：ex04 | 答案程式：`ZR_TR04_ITAB_BASICS`

本講重點

- 三個角色的分工：資料庫表、internal table（多筆）、work area（一筆）
- `TYPES ... TYPE STANDARD TABLE OF` 定義表格型別
- `APPEND` 加資料、`LOOP AT ... INTO` 逐筆處理
- `READ TABLE ... WITH KEY / INDEX` 讀單筆
- **鐵律：讀完就檢查 `sy-subrc`**；認識 `sy-tabix`
- `CLEAR` / `REFRESH` / `FREE` 的差別

1. 三個角色

角色	位置	存幾筆	比喻
資料庫表（DB table）	資料庫，永久保存	全公司的資料	倉庫
internal table（內表）	程式記憶體，程式結束就消失	這次要處理的多筆	推車
work area（工作區）	程式記憶體	**一次一筆**	手上拿的那一件

典型報表流程：把資料庫的資料 SELECT 進 internal table（講義 6），在記憶體加工，再輸出。work area 是與 internal table 之間搬進搬出的「單筆容器」，型別就是上一講的結構。

2. 宣告 internal table

先用 TYPES 定義「一列的長相」（結構），再定義「很多列」（表格型別），最後 DATA 出實體：

```
TYPES: BEGIN OF ty_student,
         id    TYPE c LENGTH 5,
         name  TYPE string,
         score TYPE i,
       END OF ty_student.

* 表格型別：ty_student 的多筆集合
TYPES tt_student TYPE STANDARD TABLE OF ty_student WITH NON-UNIQUE KEY id.

DATA: gt_students TYPE tt_student,     " internal table：多筆
      gs_student  TYPE ty_student.     " work area：一筆
```

`WITH NON-UNIQUE KEY id` 指定表格的鍵欄位（可多欄）。三種表格類別先認識、本課程用 STANDARD：

類別	特性	適用
`STANDARD TABLE`	依附加順序排列，用 INDEX 或循序搜尋	一般報表（本課程）
`SORTED TABLE`	永遠依 KEY 排序，二分搜尋	需要一直用 KEY 查
`HASHED TABLE`	雜湊，KEY 查詢速度與筆數無關，**必須 UNIQUE KEY**	大量單筆查詢

3. APPEND：加一筆到表尾

```
gs_student-id = 'S0001'. gs_student-name = '王小明'. gs_student-score = 85.
APPEND gs_student TO gt_students.

gs_student-id = 'S0002'. gs_student-name = '李小美'. gs_student-score = 92.
APPEND gs_student TO gt_students.
```

APPEND 是把 work area 的內容**複製**到表尾——APPEND 之後再改 work area，表裡那筆不會跟著變。也因此，若填新一筆時沒有把每個欄位都覆寫，舊值就會殘留混入（必要時先 `CLEAR gs_student.`）。

新式寫法對照（7.40 之後）：`VALUE` 建表、`APPEND VALUE #( )`

```
* 傳統寫法：填 work area，再 APPEND
gs_student-id = 'S0001'. gs_student-name = '王小明'. gs_student-score = 85.
APPEND gs_student TO gt_students.
gs_student-id = 'S0002'. gs_student-name = '李小美'. gs_student-score = 92.
APPEND gs_student TO gt_students.

* 新式寫法一：一次建好整張表（原本的內容會被取代）
gt_students = VALUE #( ( id = 'S0001' name = '王小明' score = 85 )
                       ( id = 'S0002' name = '李小美' score = 92 ) ).

* 新式寫法二：加一筆到表尾，不需要 work area
APPEND VALUE #( id = 'S0003' name = '陳大文' score = 67 ) TO gt_students.
```

語法拆解：

- 建內表時，`VALUE #( ... )` 裡**每一列用一組小括號包起來**：`( 欄位 = 值 ... )`，有幾列就寫幾組。講義 3 的 `VALUE #( 欄位 = 值 )` 沒有內層括號，那是建一個結構；多一層括號就是建一張表。
- `#` 的型別取自左邊的目標；用行內宣告 `DATA(gt_students) = VALUE tt_student( ... ).` 時，左邊還沒有型別，所以要明確寫出表格型別 `tt_student`。
- 新式寫法沒有 work area，所以也不會有「上一筆殘留值」的問題：每一列沒寫到的欄位都是初始值。

4. LOOP AT … INTO：逐筆處理

```
LOOP AT gt_students INTO gs_student.
  WRITE: / sy-tabix, gs_student-id, gs_student-name, gs_student-score.
ENDLOOP.
```

- 每一圈把當前那列**複製**進 work area。改 `gs_student` 不會改到表格本身（想改表格要用 MODIFY，講義 5；或 ASSIGNING，講義 16）。
- `sy-tabix`：目前處理到第幾筆（1 起算），LOOP 中隨時可用。
- 可加條件：`LOOP AT gt_students INTO gs_student WHERE score >= 60.`
- 一筆都沒迴圈到時，ENDLOOP 之後 `sy-subrc <> 0`。

新式寫法對照（7.40 之後）：`INTO DATA( )`

```
* 傳統寫法：先宣告 work area
DATA gs_student TYPE ty_student.
LOOP AT gt_students INTO gs_student.
  ...
ENDLOOP.

* 新式寫法：在 LOOP 這裡才宣告
LOOP AT gt_students INTO DATA(gs_student).
  ...
ENDLOOP.
```

`INTO DATA(gs_student)` 的型別自動取內表的列型別，效果跟傳統寫法相同（行內宣告見講義 2）。同一個程式單元裡，同一個名稱只能行內宣告一次：第二個 `LOOP` 要嘛換名稱，要嘛直接寫 `INTO gs_student`。

5. READ TABLE：讀單筆

```
* 方式一：WITH KEY──依條件找第一筆符合的
READ TABLE gt_students INTO gs_student WITH KEY id = 'S0002'.
IF sy-subrc = 0.
  WRITE: / '找到：', gs_student-name.
ENDIF.

* 方式二：INDEX──直接讀第 n 筆
READ TABLE gt_students INTO gs_student INDEX 3.
IF sy-subrc = 0.
  WRITE: / '第三筆：', gs_student-name.
ENDIF.
```

補充用法：只想知道「有沒有這筆」、不需要內容時，用 `TRANSPORTING NO FIELDS`（不搬資料，較快；找到的位置在 `sy-tabix`）：

```
READ TABLE gt_students TRANSPORTING NO FIELDS WITH KEY id = 'S0002'.
IF sy-subrc = 0.
  WRITE: / '存在，位於第', sy-tabix, '筆'.
ENDIF.
```

新式寫法對照（7.40 之後）：表格運算式 `itab[ ]` 與 `line_exists( )`

```
* 傳統寫法
READ TABLE gt_students INTO gs_student WITH KEY id = 'S0002'.
IF sy-subrc = 0.
  WRITE: / '找到：', gs_student-name.
ENDIF.

READ TABLE gt_students INTO gs_student INDEX 3.

READ TABLE gt_students TRANSPORTING NO FIELDS WITH KEY id = 'S0002'.
IF sy-subrc = 0.
  ...
ENDIF.

* 新式寫法
IF line_exists( gt_students[ id = 'S0002' ] ).        " 先確認有這筆
  gv_name = gt_students[ id = 'S0002' ]-name.         " 直接取那一列的欄位
ENDIF.

gs_student = gt_students[ 3 ].                         " 第 3 列

IF line_exists( gt_students[ id = 'S0002' ] ).        " 只問有沒有
  ...
ENDIF.
```

語法拆解：

- `內表[ 條件 ]` 叫表格運算式，本身就代表「符合條件的那一列」，可以整列賦值、也可以直接接 `-欄位` 取單一欄位。中括號裡寫 `欄位 = 值`（等於 `WITH KEY`），或直接寫數字（等於 `INDEX`）。
- **找不到時不會設定 `sy-subrc`，而是發生例外 `CX_SY_ITAB_LINE_NOT_FOUND`**，沒有處理的話程式會 dump（2026-10-08 實測）。所以要先用 `line_exists( 內表[ 條件 ] )` 判斷有沒有這筆，它回傳真 / 假，可以直接放在 `IF` 後面，取代 `TRANSPORTING NO FIELDS` 的用法。
- 也可以寫 `gs_student = VALUE #( gt_students[ id = 'S9999' ] OPTIONAL ).`：找不到時回傳空的一列，不會發生例外。
- 什麼時候用哪個：只要拿一個欄位、或要放在運算式裡 → 表格運算式；同一筆要取好幾個欄位，或要知道 `sy-tabix` → `READ TABLE` 一樣好用。

練習 4 的新式寫法版本見答案程式 `ZR_TR04_ITAB_BASICS_NEW`。輸出跟原版逐行比對，只有最後一行不同：原版印 `sy-subrc = 4`，新版改印 `line_exists( ) = false`，正是上面說的「表格運算式不設 `sy-subrc`」。

6. 鐵律：檢查 sy-subrc

`READ TABLE` 找不到時**不會報錯、不會當掉**，只是 `sy-subrc` 不為 0，而且 **work area 保持原內容不變**。不檢查 sy-subrc 的後果：

```
READ TABLE gt_students INTO gs_student WITH KEY id = 'S0002'.   " 找到，gs 是李小美
READ TABLE gt_students INTO gs_student WITH KEY id = 'S9999'.   " 沒找到！
WRITE gs_student-name.    " 印出來還是「李小美」——上一筆的殘留值！
```

這就是實務上「查無資料卻顯示了別人的資料」這類 bug 的標準成因。養成反射動作：**READ TABLE 下一行永遠是 `IF sy-subrc = 0.`**。同理適用之後所有會設定 sy-subrc 的指令（SELECT、CALL FUNCTION…）。

7. CLEAR / REFRESH / FREE

指令	對 work area	對 internal table
`CLEAR gs.`	欄位全部歸初始值	-
`CLEAR gt.` / `REFRESH gt.`	-	刪光所有列（記憶體保留）
`FREE gt.`	-	刪光所有列並釋放記憶體

日常用 `CLEAR` 就夠；`REFRESH` 是舊寫法，讀舊程式會遇到，效果同 `CLEAR gt.`。

8. 常見錯誤與陷阱

症狀	原因
查無資料卻顯示出值	READ 失敗沒檢查 sy-subrc，用到 work area 殘留值
兩筆資料長一樣	APPEND 後忘了改（或 CLEAR）work area 又 APPEND 一次
LOOP 裡改了值，表格沒變	INTO 是複本，要 MODIFY（講義 5）或 ASSIGNING（講義 16）
`sy-tabix` 在 LOOP 外不對	它只在 LOOP / READ 的當下有意義，別事後才用
READ INDEX 0 或超過筆數	sy-subrc = 4/8，一樣要檢查

9. 課堂練習

完成 ex04：建學生內表、APPEND 三筆、LOOP 輸出、WITH KEY 與 INDEX 讀取，並實際觀察 sy-subrc 與殘留值現象。